Documentation de l'Extension : Représentation Intermédiaire pour une compilation RISC-V

Projet Génie Logiciel

Équipe gl25
Alexandre Danhiez, Noah Deplace, Ilès Ouyahia,
Nicolas Montauriol, Nathan Vigouroux

Janvier 2026

Ensimag

Table des matières

1 Introduction **3**
- 1.1 Contexte du projet : Le compilateur Deca de base . . . 3
- 1.2 Problématique : IMA vs RISC-V et Modularité . . . 3
- 1.3 Objectifs de l'extension . . . 3
  - 1.3.1 Introduction d'une Représentation Intermédiaire (IR) . . . 4
  - 1.3.2 Modularité et Backends Multiples . . . 4
  - 1.3.3 Périmètre de l'extension . . . 4

2 Analyse et État de l'art **5**
- 2.1 Architecture des compilateurs modernes . . . 5
  - 2.1.1 Frontend : Analyse et Abstraction . . . 5
  - 2.1.2 Middle-end : Optimisation et IR . . . 5
  - 2.1.3 Backend : Spécialisation . . . 5
- 2.2 Les Représentations Intermédiaires (3AC) . . . 6
  - 2.2.1 L'approche CISC vs RISC . . . 6
  - 2.2.2 Le Three-Address Code (3AC) . . . 6
- 2.3 Présentation des nouvelles cibles . . . 7
  - 2.3.1 RISC-V : Architecture Load/Store moderne . . . 7
  - 2.3.2 QBE : Backend Universel . . . 7

3 Spécification de l'extension **8**
- 3.1 Périmètre fonctionnel . . . 8
  - 3.1.1 Fonctionnalités supportées . . . 8
  - 3.1.2 Fonctionnalités exclues . . . 8
- 3.2 Définition formelle de l'IR implémentée . . . 8
  - 3.2.1 Philosophie : 3AC avec Load/Store explicite . . . 8
  - 3.2.2 Structure d'un Programme IR . . . 9
  - 3.2.3 Types de données et Opérandes . . . 9
  - 3.2.4 Jeu d'Instructions (ISA) . . . 9
- 3.3 Interface des Backends . . . 10
  - 3.3.1 Backend RISC-V : Gestion explicite des ressources . . . 10
  - 3.3.2 Backend QBE : Délégation de l'allocation . . . 11
  - 3.3.3 Backend IMA : L'héritage . . . 11

4 Conception et Architecture Logicielle **12**
- 4.1 Architecture globale : Flux de données . . . 12
- 4.2 Le package `fr.ensimag.deca.ir` . . . 12
  - 4.2.1 Modélisation de l'Instruction . . . 12
  - 4.2.2 Typage fort des Opérandes . . . 13
  - 4.2.3 Structure du Programme IR . . . 13
- 4.3 Génération de l'IR (`IRGenerator`) . . . 14
  - 4.3.1 Stratégie de visite . . . 14
  - 4.3.2 Exemple d'implémentation : L'Addition . . . 14
  - 4.3.3 Gestion du contexte . . . 14
- 4.4 L'interface des Backends (`IRVisitor`) . . . 15
  - 4.4.1 Le contrat `IRVisitor` . . . 15
  - 4.4.2 Architecture interne du Backend RISC-V . . . 15

5 Algorithmes et Choix d'Implémentation **16**
5.1 Traduction des expressions (AST vers IR) . . . 16
5.1.1 Décomposition des expressions . . . 16
5.1.2 Gestion des registres virtuels . . . 16
5.2 Traduction des structures de contrôle . . . 16
5.3 Traduction des structures de contrôle . . . 16
5.3.1 Traduction optimisée du While . . . 16
5.4 Génération de code cible . . . 17
5.4.1 Sélection d'instructions . . . 17
5.4.2 Implémentation de la stratégie "All-on-Stack" . . . 17
5.4.3 Différences entre Simulateur et Bare-Metal . . . 18

6 Environnement et Validation **19**
6.1 Environnement Docker . . . 19
6.1.1 Construction de l'image . . . 19
6.1.2 Cycle de vie et Configuration . . . 19
6.2 Stratégie de test multiniveaux . . . 20
6.2.1 Niveau 1 : Tests Unitaires de l'IR (White Box) . . . 20
6.2.2 Niveau 2 : Tests d'Intégration (Black Box) . . . 21
6.3 Résultats . . . 22
6.3.1 Couverture fonctionnelle . . . 22
6.4 Limites de la validation . . . 22

7 Conclusion et Perspectives **23**
7.1 Bilan technique . . . 23
7.2 Bilan pédagogique . . . 23
7.3 Perspectives d'évolution . . . 24
7.3.1 Support de Deca Objet et VTables . . . 24
7.3.2 Optimisations sur l'IR . . . 24
7.3.3 Évolutivité du Backend RISC-V . . . 24

1 Introduction

Le projet Génie Logiciel de deuxième année à l'Ensimag a pour objectif principal la conception et l'implémentation d'un compilateur complet pour le langage *Deca*, un sous-langage de Java.

Dans ce projet, l'architecture cible est imposée : il s'agit de la machine abstraite type 68000 **IMA**. Bien que pédagogique, cette contrainte architecturale masque certains enjeux majeurs de la compilation moderne, notamment la portabilité et l'optimisation indépendante de la cible.

Ce document présente l'extension réalisée par l'équipe **gl25**, qui vise à transformer l'architecture monolithique du compilateur `decac` en une architecture modulaire basée sur une **Représentation Intermédiaire (IR)**. Cette refonte permet de supporter plusieurs backends, dont l'assembleur **RISC-V**, standard industriel ouvert, ainsi que le backend générique **QBE**.

1.1 Contexte du projet : Le compilateur Deca de base

Le compilateur `decac`, tel que spécifié dans le sujet initial, suit une architecture classique découpée en trois passes principales :

1. **L'étape A (Lexique et Syntaxe) :** Elle utilise *ANTLR* pour transformer le flux de caractères en un Arbre Syntaxique Abstrait (AST). Cette étape valide la structure grammaticale du programme.
2. **L'étape B (Vérifications Contextuelles) :** Elle parcourt l'AST pour décorer les nœuds avec des types et vérifier la cohérence sémantique (typage, héritage, ...).
3. **L'étape C (Génération de Code) :** C'est l'étape finale où l'AST décoré est parcouru pour émettre directement les instructions assembleur.

Dans l'implémentation standard attendue pour le projet, l'étape de génération de code est **fortement couplée** à l'architecture cible IMA. Les visiteurs parcourant l'arbre manipulent directement des concepts propres à cette machine, tels que les registres physiques spécifiques (`R0, R1`) ou les offsets de pile fixes.

1.2 Problématique : IMA vs RISC-V et Modularité

La machine IMA possède des particularités architecturales marquées (modèle CISC). Elle autorise notamment l'utilisation d'opérandes mémoire directement dans les instructions arithmétiques, ce qui simplifie la génération de code mais "cache" les accès mémoire réels.

À l'inverse, **RISC-V** est une architecture *Load/Store* stricte : les opérations ne se font qu'entre registres, et les accès mémoire sont explicites. Elle repose également sur un grand nombre de registres généraux (32) qu'il faut gérer manuellement.

Adapter le code existant pour supporter RISC-V demanderait de dupliquer la totalité des méthodes `codegen`, violant ainsi le principe DRY (*Don't Repeat Yourself*) et rendant la maintenance du projet périlleuse.

1.3 Objectifs de l'extension

Pour répondre à ces problématiques, notre extension se propose de restructurer la fin de la chaîne de compilation en introduisant une couche d'abstraction supplémentaire.

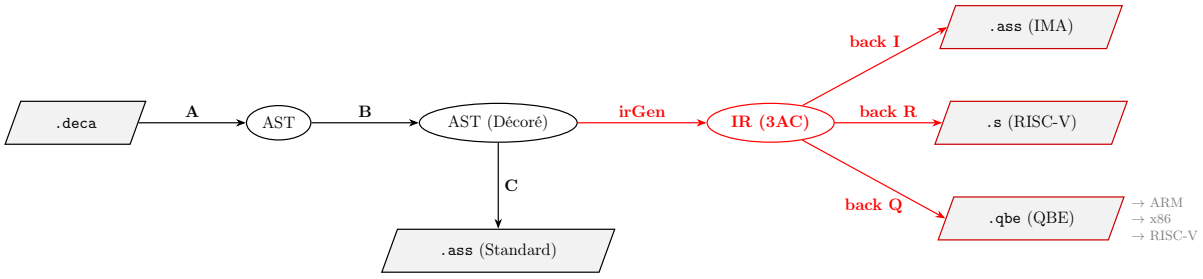


FIGURE 1.1 – Flux de compilation : legacy en noir et avec IR en rouge.

1.3.1 Introduction d'une Représentation Intermédiaire (IR)

L'objectif central est la conception et l'implémentation d'une **Représentation Intermédiaire** (IR). Cette IR se situe entre l'AST et le code machine. Nous avons fait le choix d'une IR de type **Three-Address Code (3AC)**, structurée linéairement.

Cette représentation possède les caractéristiques suivantes :

— Elle est **linéaire** (une suite d'instructions) et non arborescente.

— Elle utilise un nombre illimité de **registres virtuels**, ce qui sépare le problème de la sélection d'instructions de celui de l'allocation de registres (spécifique à l'archi).

1.3.2 Modularité et Backends Multiples

Grâce à cette IR, nous visons l'implémentation de trois *backends* distincts, prouvant ainsi la modularité de notre approche :

— **Backend RISC-V :** Cible principale de l'extension. Il permet d'exécuter les programmes Deca sur un simulateur standard (RARS) ou potentiellement sur du matériel réel. Cela nous confronte aux contraintes réelles des architectures modernes.

— **Backend IMA :** Ré-implémentation du support IMA à partir de notre IR. Cela permet de valider que notre IR est suffisamment générique pour supporter l'architecture historique du projet.

— **Backend QBE :** Génération de code pour *QBE* (Quick BackEnd), un backend de compilateur léger écrit en C. Cela démontre la capacité de notre IR à s'interfacer avec d'autres chaînes de compilation existantes.

1.3.3 Périmètre de l'extension

Compte tenu de la complexité inhérente à l'ajout d'une couche d'abstraction complète, nous avons restreint le périmètre fonctionnel de cette extension au langage **Deca sans objet**. L'extension supporte donc les types de base (`int`, `float`, `boolean`), les structures de contrôle (`while`, `if`, `else`), les opérations arithmétiques et logiques, ainsi que les entrées/sorties. L'implémentation de la gestion dynamique de la mémoire (Tas) et des tables de méthodes virtuelles pour la partie Objet reste une perspective d'évolution.

2 Analyse et État de l'art

Avant d'entamer la conception de notre extension, il est nécessaire de comprendre les spécificités de l'architecture cible historique (IMA) et d'étudier les solutions standards utilisées dans l'industrie des compilateurs. Ce chapitre présente l'architecture modulaire moderne, justifie le choix du *Three-Address Code* et analyse les différences fondamentales entre les cibles visées.

2.1 Architecture des compilateurs modernes

L'architecture traditionnelle d'un compilateur, telle qu'implémentée dans des projets d'envergure comme **GCC** ou **LLVM**, repose sur une séparation stricte des responsabilités en trois étages. Cette séparation vise à résoudre le problème de la complexité croissante des langages et de la diversité matérielle.

2.1.1 Frontend : Analyse et Abstraction

Le Frontend est spécifique au langage source (ici, Deca). Son rôle est de valider le code par l'analyse lexicale et syntaxique, puis de vérifier la sémantique (typage, héritage, ...). Le produit de cette étape est généralement un Arbre Syntaxique Abstrait (AST). Dans notre projet, cette partie correspond aux étapes A et B déjà implémentées.

2.1.2 Middle-end : Optimisation et IR

C'est le composant central ajouté par notre extension. Le Middle-end travaille sur une **Représentation Intermédiaire (IR)**. Cette représentation est indépendante du langage source et de la machine cible.

L'objectif du Middle-end est de préparer le code pour la génération :

— **Simplification** des structures de contrôle complexes (boucles, branchements).

— **Atomisation** des opérations telles que : `a = b*5 + c ;`

— **Optimisations** indépendantes de la machine comme la propagation de constantes.

L'utilisation d'une IR permet de factoriser ces opérations : elles sont écrites une seule fois, quelle que soit la cible finale.

2.1.3 Backend : Spécialisation

Le Backend est spécifique à l'architecture cible (RISC-V, IMA, QBE). Il reçoit l'IR et effectue deux tâches critiques :

— **Sélection d'instructions :** Traduire les opérations génériques de l'IR en instructions spécifiques à la machine cible.

— **Allocation de registres :** Mapper les variables temporaires de l'IR vers le nombre fini de registres physiques du processeur cible.

2.2 Les Représentations Intermédiaires (3AC)

Le choix de la forme de l'IR est déterminant pour combler le fossé sémantique entre l'AST (haut niveau) et l'assembleur.

2.2.1 L'approche CISC vs RISC

L'architecture **IMA** est une machine abstraite inspirée du Motorola 68000. C'est une architecture de type **CISC** (Complex Instruction Set Computer) à deux adresses. Elle permet d'utiliser des opérandes mémoire directement dans les instructions arithmétiques. Par exemple, l'instruction suivante est valide en IMA :

$$\texttt{ADD 3(GB), R2} \quad (\text{R2} \leftarrow \text{R2} + \text{Mémoire}[GB + 3])$$

À l'inverse, les architectures modernes comme **RISC-V** imposent une contrainte **Load/Store** : les opérations arithmétiques ne peuvent se faire qu'entre registres. L'opération ci-dessus nécessiterait deux instructions distinctes en RISC-V (un chargement `LW`, puis une addition `ADD`).

Cette différence fondamentale rend la génération de code monolithique difficile à adapter : le générateur pour IMA "cache" la complexité des accès mémoire, ce qui n'est pas possible pour RISC-V.

2.2.2 Le Three-Address Code (3AC)

Pour notre extension, nous avons opté pour une IR de type **Three-Address Code** [4]. Dans cette représentation, chaque instruction possède au plus trois opérandes (généralement deux sources et une destination) et décompose les opérations complexes en opérations atomiques de la forme :

$$x = y \ \textbf{op} \ z$$

Contrairement à l'AST qui est hiérarchique, le 3AC est une liste linéaire d'instructions.

Exemple de transformation : L'expression Deca `a = b + c * d;` se traduit ainsi, en supposant que `b`, `c`, `d` sont en mémoire :

Approche CISC (IMA)	**Approche 3AC (Notre IR)**
(Utilisation d'opérandes mémoire)	*(Décomposition explicite)*
`LOAD c, R2`	`temp1 = c * d`
`MUL d, R2`	`temp2 = b + temp1`
`ADD b, R2`	`a = temp2`
`STORE R2, a`	

Cette forme manipule des registres virtuels (`tempX`), laissant au Backend le choix de l'allocation physique.

2.3 Présentation des nouvelles cibles

2.3.1 RISC-V : Architecture Load/Store moderne

RISC-V est une architecture de jeu d'instructions (ISA) standard ouverte [2]. Contrairement à IMA, c'est une architecture purement RISC :

— **Architecture Load/Store :** Seules les instructions `load` et `store` accèdent à la mémoire. Toutes les autres opérations (add, sub, mul) se font exclusivement entre registres.

— **Registres nombreux :** Elle dispose de 32 registres (`x0` à `x31`), contre 16 pour IMA.

Le défi pour l'implémentation RISC-V, c'est d'expliciter tous les chargements mémoire implicites et de gérer l'allocation des 32 registres efficacement.

2.3.2 QBE : Backend Universel

QBE (Quick Backend) est un compilateur backend léger [1]. Il fournit un language intermédiaire textuelle (IL) de bas niveau. L'intérêt d'intégrer QBE est double :

— Il agit comme une "preuve de concept" de la généricité de notre propre IR : si nous pouvons traduire notre 3AC vers l'IL de QBE, nous validons notre design.

— Il ouvre notre compilateur au monde : QBE sait générer de l'assembleur optimisé pour x86-64, ARM64 et RISC-V. En ciblant QBE, `decac` devient *de facto* multi-plateforme.

3 Spécification de l'extension

Ce chapitre définit formellement le périmètre fonctionnel couvert par l'extension, ainsi que la syntaxe et la sémantique de la Représentation Intermédiaire (IR) conçue pour assurer la modularité du compilateur.

3.1 Périmètre fonctionnel

L'extension se concentre sur le support d'un sous-ensemble strict du langage Deca, désigné ici par **Deca "Sans Objet"**. L'objectif est de valider la chaîne de compilation modulaire sur les constructions algorithmiques fondamentales avant d'aborder la complexité liée à la gestion dynamique de la mémoire objet.

3.1.1 Fonctionnalités supportées

Le compilateur étendu supporte la compilation et l'exécution correcte des constructions suivantes :

- **Types primitifs :** Support complet des types `int`, `float` et `boolean`.
- **Déclarations de variables :** Déclaration avec ou sans initialisation, portée locale.
- **Arithmétique :** Les 5 opérations binaires (`+`, `-`, `*`, `/`, `%`) sur les entiers et les flottants.
- **Logique et Comparaisons :** Opérateurs booléens (`&&`, `||`, `!`) et de comparaison (`==`, `!=`, `<`, `>`, `<=`, `>=`).
- **Structures de contrôle :** Blocs conditionnels `if`, `else if`, `else` et boucles `while`.
- **Sorties :** Instructions `print`, `println` pour tous les types supportés.
- **Conversions implicites :** La promotion arithmétique automatique de `int` vers `float` est supportée (ex : `float x = 1 + 2.5;`).

3.1.2 Fonctionnalités exclues

Pour des questions de temps, les éléments suivants sont exclus de cette première version de l'IR :

- **Programmation Orientée Objet :** Pas de classes, pas de méthodes, pas d'héritage, pas d'instanciation.
- **Casts explicites avec perte de précision :** Le cast explicite `(int) float` n'est pas supporté par l'IR actuelle.
- **IO limité :** `printx` et `printlnx` non implémentés. La conversion en string hexadécimale étant relativement haut niveau, il est peu pratique et peu intéressant d'implémenter cette fonctionnalité. De même pour `readint` et `readfloat`.

3.2 Définition formelle de l'IR implémentée

Notre Représentation Intermédiaire est une forme de **Three-Address Code (3AC) linéaire**, enrichie d'instructions de gestion mémoire explicites.

3.2.1 Philosophie : 3AC avec Load/Store explicite

Dans un compilateur industriel (type GCC), l'IR est souvent purement virtuelle (registres infinis) et c'est le backend qui décide quand "spiller" (déverser) les registres en mémoire. Compte

tenu des contraintes temporelles fortes du projet, nous avons choisi de simplifier les backends en rendant la gestion de la mémoire **explicite dès l'IR**.

L'IR distingue donc deux espaces de stockage :

1. **La Stack (Mémoire) :** Où vivent les variables déclarées dans le programme source (ex : `var_a`, `var_b`).
2. **Les Registres Virtuels :** Où s'effectuent tous les calculs (ex : `s0`, `s1`, ...).

Ainsi, une addition `a = b + c` ne se traduit pas directement par `ADD a, b, c` mais par une séquence `LOAD -> ADD -> STORE`. Cela garantit que les backends n'ont pas à gérer d'algorithmes complexes d'allocation de registres pour produire du code fonctionnel.

3.2.2 Structure d'un Programme IR

Un programme IR est structuré en quatre segments distincts, facilitant la traduction vers l'assembleur (notamment RISC-V et x86) :

`.data` **Constantes globales :** Contient les chaînes de caractères statiques (pour les print) et les éventuelles tables globales.

`.stack` **Variables locales :** Déclare la liste des variables du programme et leur type (`INT`, `FLOAT`). Ce segment permet au backend de calculer la taille de la "Stack Frame" à allouer en début de fonction.

`.heap` **Tas dynamique :** Réservé pour l'allocation dynamique (vide dans la version "Sans Objet").

`.text` **Instructions :** La séquence linéaire des instructions exécutables.

3.2.3 Types de données et Opérandes

Les instructions manipulent trois types d'opérandes :

— **Immediate (Imm) :** Valeurs constantes (ex : `#42`, `#3.14`) avec la convention `#0` (resp. `#1`) pour `false` (resp. `true`).

— **Virtual Register (Reg) :** Registres temporaires de calcul (ex : `s0`, `s1`). Leur nombre est illimité.

— **Stack Variable (Var) :** Identifiants de variables stockées en mémoire (ex : `x: FLOAT`, `counter: INT`).

3.2.4 Jeu d'Instructions (ISA)

Table des Instructions

Toutes les instructions entières (`OP_INT`) ont un équivalent flottant (`OP_FLOAT`), à l'exception du modulo.

Catégorie	Syntaxe et Sémantique
Mémoire	`LOAD_INT Rm, Var` : $R_m \leftarrow Mem[Var]$ `STORE Rm, Var` : $Mem[Var] \leftarrow R_m$
Sauts	`JMP label` : Saut inconditionnel vers l'étiquette. `Bcc_INT Rx, Ry, label` : Saut conditionnel. *Variantes :* `BEQ` ($=$), `BNE` ($\neq$), `BLT` ($<$), `BLE` ($\leq$), `BGT` ($>$), `BGE` ($\geq$).
Arithmétique	`ADD_INT dst, lhs, rhs` : $dst \leftarrow lhs + rhs$ `SUB_INT dst, lhs, rhs` : $dst \leftarrow lhs - rhs$ `MUL_INT dst, lhs, rhs` : $dst \leftarrow lhs \times rhs$ `DIV_INT dst, lhs, rhs` : $dst \leftarrow lhs/rhs$ `MOD_INT dst, lhs, rhs` : $dst \leftarrow lhs\%rhs$
Unaire	`CONV_FLOAT dst, src` : $dst \leftarrow (float)src$ `NEG_INT dst, src` : $dst \leftarrow -src$
Entrées/Sorties	`PRINT_INT src` : Affiche l'entier contenu dans *src*. `PRINT_FLOAT src` : Affiche le flottant contenu dans *src*. `PRINT_CHAR label` : Affiche la chaîne constante stockée en `.data` à l'adresse `label`.

3.3 Interface des Backends

Pour garantir la modularité, l'IR est découplée de la génération de code final via le patron de conception *Visitor*. Chaque générateur de code (Backend) implémente une interface commune mais adapte sa stratégie aux contraintes de la cible.

3.3.1 Backend RISC-V : Gestion explicite des ressources

Le générateur RISC-V cible le simulateur **RARS**. Cette cible a été choisie pour simplifier le développement initial :

- **Appels Système (ABI) :** RARS fournit une couche d'abstraction pour les entrées/sorties via l'instruction `ecall`. Par exemple, afficher un entier se fait simplement en plaçant la valeur dans `a0` et le code service `1` dans `a7`. Cela nous dispense de gérer les appels systèmes Linux complexes (POSIX *sys_ write*, gestion des buffers).
- **Gestion des registres :** Dans cette première version, nous n'implémentons pas d'allocateur de registres optimisé (type *Graph Coloring*). La stratégie adoptée est celle du "All-on-Stack" : chaque registre virtuel de l'IR correspond à un emplacement sur la pile. Les registres physiques temporaires (`t0-t6`) ne sont utilisés que très brièvement pour effectuer les opérations arithmétiques avant de renvoyer le résultat en mémoire.

Nous avons donc choisi ici une version très simplifiée de gestion des registres. Une version plus élaborée est disponible dans la chaîne de compilation legacy.

3.3.2 Backend QBE : Délégation de l'allocation

Le backend QBE cible le Langage Intermédiaire de QBE (`.qbe`). Contrairement aux autres, ce backend est de plus haut niveau :

— **Pas d'allocation de registres :** QBE acceptant une infinité de temporaires (notés `%tmp`), notre backend traduit directement `s0` en `%s0` sans se soucier des limitations physiques. C'est l'outil QBE qui fera ce travail complexe plus tard.

— **Typage explicite :** QBE exige de typer chaque instruction (ex : `add` pour les entiers 32 bits devient `addw`, pour les flottants `adds`). Le backend utilise les informations de type présentes dans nos instructions IR (`_INT` vs `_FLOAT`) pour choisir le bon suffixe.

3.3.3 Backend IMA : L'héritage

Ce backend recrée le comportement du compilateur original. Il utilise les registres `R2` à `R15` pour les calculs. Sa principale difficulté réside dans la gestion des accumulateurs limités pour les opérations complexes, résolue ici par la structure atomique du 3AC.

4 Conception et Architecture Logicielle

L'introduction d'une Représentation Intermédiaire change la structure interne du compilateur. Ce chapitre détaille les choix de conception orientés objet qui ont permis de découpler le Frontend du Backend, en s'appuyant sur les patrons de conception standards (Visitor, Composite).

4.1 Architecture globale : Flux de données

Dans la version monolithique du compilateur `decac`, le flux de contrôle est linéaire : l'étape A produit un AST, l'étape B le décore, et l'étape C parcourt cet AST pour écrire directement des chaînes de caractères dans un fichier de sortie.

Notre extension introduit un point de bifurcation à la fin de l'étape B. L'architecture globale suit désormais le patron *Pipeline*, où les données subissent des transformations successives :

1. **Frontend (Analyse) :** inchangé. Il produit un `AbstractProgram` validé sémantiquement.
2. **Middle-end (Génération IR) :** Une nouvelle classe, `IRGenerator`, parcourt l'AST. Contrairement au générateur classique, elle ne produit pas de texte, mais instancie en mémoire un objet complexe : l'`IRProgram`.
3. **Backend (Génération Cible) :** L'`IRProgram` est passé à un `CodeGenerator` spécifique (RISC-V, IMA ou QBE). Ce dernier parcourt la liste linéaire des instructions IR pour produire le fichier final.

Cette architecture assure une séparation stricte des préoccupations : le Frontend ne "sait" pas pour quelle machine il travaille, et le Backend ne "sait" pas de quelle construction Deca (While, If) provient l'instruction qu'il traduit.

4.2 Le package `fr.ensimag.deca.ir`

Le cœur de l'extension réside dans la modélisation objet de l'IR. Nous avons conçu ce package pour qu'il soit fortement typé, évitant ainsi les erreurs de manipulation d'opérandes.

4.2.1 Modélisation de l'Instruction

Contrairement à l'AST qui est une structure arborescente (Composite Pattern), l'IR est une structure séquentielle. Cependant, les instructions elles-mêmes suivent une hiérarchie de classes stricte pour permettre le polymorphisme.

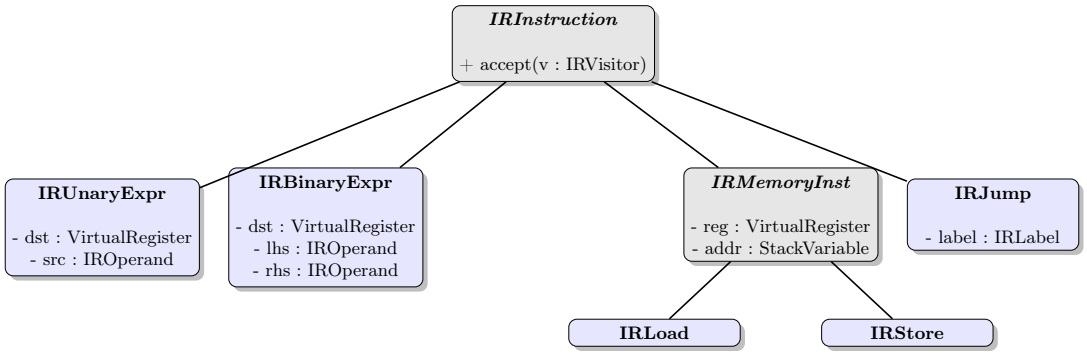


FIGURE 4.1 – Diagramme de classes simplifié de la hiérarchie des instructions IR.

Toutes les instructions héritent de la classe abstraite `IRInstruction`. Cette classe définit la méthode abstraite `accept(IRVisitor v)`, essentielle pour le traitement ultérieur.

4.2.2 Typage fort des Opérandes

Le choix de modéliser les opérandes via une hiérarchie de classes stricte répond à deux impératifs architecturaux :

1. **Sélection d'instruction par le Backend :** Certaines cibles, comme QBE, sont typées. Le générateur de code doit impérativement savoir si un opérande est un immédiat, une variable ou un registre pour générer le mnémonique correct (ex : distinguer une addition entière `addw` d'une addition flottante `adds` selon le type de l'opérande).
2. **Potentiel d'optimisation :** En conservant la nature sémantique de l'opérande (ex : savoir qu'il s'agit d'un `ImmediateInteger` de valeur 0), le backend peut effectuer des optimisations locales (ex : suppression d'une instruction `ADD x, x, #0`).

L'interface `IROperand` se décline ainsi :

— `VirtualRegister` : Représente un registre temporaire de calcul (`s0`, `s1`...). Il est identifié par un index unique.
— `Immediate` : Classe abstraite pour les constantes littérales.
  — `ImmediateInteger` : Encapsule une valeur `int`.
  — `ImmediateFloat` : Encapsule une valeur `float`.
— `StackVariable` : Représente une variable locale du programme source. Elle ne stocke pas d'adresse physique, mais une référence symbolique. C'est le Backend qui résoudra cette référence en un offset final (ex : `-4(FP)` ou `local_var` en QBE).

4.2.3 Structure du Programme IR

La classe conteneur `IRProgram` structure le code en segments logiques distincts, imitant la structure d'un binaire ou d'un fichier assembleur. Elle agit également comme une usine pour les ressources temporaires (registres, labels).

```
public class IRProgram {
    // Segments de données et de code
    private final LinkedList<IRData> data = new LinkedList<>();
    private final LinkedList<IRAbstractInst> text = new LinkedList<>();

    // Gestion de la mémoire (Tables des symboles)
    // LinkedHashMap est utilisé pour préserver l'ordre de déclaration
    private final LinkedHashMap<String, IRVariable> stack = new LinkedHashMap
    <>();
    private final LinkedHashMap<String, IRVariable> heap = new LinkedHashMap<>()
    ;

    // Pool de chaines de caractères (pour éviter les doublons)
    private final Map<String, IRLabel> labels = new HashMap<>();

    // Compteurs pour la génération unique
    private int sid = 0; // Pour les registres "Scratch"


    // ...

}
```

Cette conception centralisée offre plusieurs avantages :

— **Gestion automatique des ressources :** C'est l'`IRProgram` qui garantit l'unicité des registres virtuels (`sid`) et des labels via ses méthodes de fabrication.

— **String Pooling :** La map `String` évite de dupliquer les constantes chaînes de caractères identiques dans le segment `.data`.
— **Ordre des variables :** L'utilisation de `LinkedHashMap` pour le segment `stack` permet au Backend de parcourir les variables dans l'ordre de leur déclaration, facilitant le calcul déterministe des offsets dans la stack frame.

Cette séparation permet au backend RISC-V, par exemple, de générer toute la section `.data` avant de commencer à écrire le code dans `.text`.

4.3 Génération de l'IR (`IRGenerator`)

La classe `IRGenerator` est responsable de la traduction $AST \rightarrow IR$. Elle implémente une visite de l'arbre syntaxique décoré.

4.3.1 Stratégie de visite

Nous utilisons un parcours en profondeur (Post-Order Traversal) pour les expressions. L'idée est la suivante : pour compiler un nœud, il faut d'abord avoir compilé ses enfants et récupéré les registres contenant leurs résultats.

La méthode de visite retourne un `VirtualRegister` contenant le résultat de l'opération du sous-arbre.

Cela est similaire au fonctionnement de `genCode` du compilateur legacy.

4.3.2 Exemple d'implémentation : L'Addition

Voici un extrait simplifié de la méthode gérant le nœud `Plus` de l'AST Deca. On note l'utilisation du gestionnaire de registres (`RegisterManager`) pour obtenir de nouveaux temporaires.

```
@Override
public VirtualRegister visitPlus(Plus node) {
    // 1. Visite récursive des opérandes gauche et droite
    // Chaque visite retourne le registre virtuel contenant le résultat
    VirtualRegister lhsReg = node.getLeftOperand().accept(this);
    VirtualRegister rhsReg = node.getRightOperand().accept(this);

    // 2. Allocation d'un nouveau registre pour le résultat
    VirtualRegister dstReg = this.regManager.newVirtualRegister();

    // 3. Création de l'instruction IR correspondante
    // Note : On gère ici le typage (Int vs Float)
    IRInstruction inst;
    if (node.getType().isInt()) {
        inst = new ADD_INT(dstReg, lhsReg, rhsReg);
    } else {
        inst = new ADD_FLOAT(dstReg, lhsReg, rhsReg);
    }

    // 4. Ajout au programme courant
    this.currentProgram.addInstruction(inst);

    // 5. On retourne le registre résultat pour le parent
    return dstReg;
}
```

4.3.3 Gestion du contexte

Le générateur maintient un état interne comprenant :
— Un **compteur de registres** : Assure l'unicité des noms (`s0`, `s1`...).

— Un **générateur de labels** : Fournit des labels uniques pour les branchements (`if_start_42`, `while_cond_12`).

4.4 L'interface des Backends (`IRVisitor`)

Une fois l'objet `IRProgram` construit en mémoire, il doit être traduit en fichier texte. Pour cela, nous utilisons le pattern **Visitor** sur la structure de l'IR elle-même.

4.4.1 Le contrat `IRVisitor`

L'interface `IRVisitor` définit une méthode pour chaque instruction concrète de l'IR.

```
public interface IRVisitor {
    // Methods pour chaque type d'instruction IR
    void visit(IRAssign inst);
    void visit(IRLoad inst);
    void visit(IRPrintValue inst);
    void visit(IRBinaryExpr inst);
    void visit(IRPrintString inst);
}
```

Cette interface force tous les backends à traiter l'intégralité du jeu d'instructions. Si nous ajoutons une instruction à l'IR, le compilateur Java refusera de compiler tant que tous les backends n'auront pas implémenté la nouvelle méthode, garantissant la robustesse du code.

4.4.2 Architecture interne du Backend RISC-V

Pour gérer la complexité de la génération d'assembleur (gestion de la pile, formatage, erreurs), nous n'avons pas tout implémenté dans une classe monolithique `RVGenerator`. Au contraire, nous avons adopté une architecture modulaire articulée autour d'un contexte partagé (`RVContext`).

L'objet `RVGenerator` orchestre une suite de composants spécialisés (Managers) :

— `RVStackManager` **:** Responsable du calcul de la *Stack Frame*. Il associe chaque variable virtuelle IR à un offset physique (via la classe `RVOffsets`). C'est lui qui sait que la variable `x` est à `12(sp)`.

— `RVDataManager` **:** Gère les mouvements de données. Il encapsule la complexité du chargement des opérandes (immédiats, registres, mémoire) vers les registres physiques via des visiteurs internes (`DValVisitor`). **Cette abstraction est cruciale :** elle isole la logique de "comment charger une donnée" de "quelle donnée charger". Un spécialiste RISC-V pourrait ainsi réécrire ce Manager pour implémenter un algorithme de *Spilling* complexe ou utiliser des instructions vectorielles, sans avoir à modifier une seule ligne de la logique de parcours de l'IR dans `RVVisitor`.

— `RVErrorManager` **:** Centralise la génération du code de gestion d'erreur (division par zéro, débordements) et l'émission des données statiques associées (messages d'erreur).

— `RVFormatter` **:** Abstraction de bas niveau pour l'écriture du texte assembleur. Il gère l'indentation, les directives (`.data`, `.text`) et l'échappement des chaînes.

L'objet `RVContext` agit comme un point d'accès unique (Façade) permettant à ces managers de communiquer entre eux sans couplage fort.

5 Algorithmes et Choix d'Implémentation

La génération de code via une représentation intermédiaire nécessite deux phases de traduction distinctes : l'aplatissement de l'arbre syntaxique vers l'IR, puis la sélection d'instructions et l'allocation de ressources vers la cible finale. Ce chapitre détaille les algorithmes mis en œuvre pour ces deux étapes.

5.1 Traduction des expressions (AST vers IR)

L'objectif de cette phase est de transformer une structure hiérarchique (l'AST) en une séquence linéaire d'instructions élémentaires.

5.1.1 Décomposition des expressions

Nous utilisons un parcours récursif de l'AST. Pour chaque nœud expression (binaire ou unaire), l'algorithme suit le schéma suivant :

1. Visiter récursivement les opérandes gauche et droit. Ces visites génèrent les instructions nécessaires pour calculer les sous-expressions et retournent les registres virtuels contenant leurs résultats temporaires.
2. Allouer un nouveau registre virtuel pour le résultat courant.
3. Générer l'instruction IR effectuant l'opération (ex : `ADD_INT`) entre les registres des opérandes et le registre résultat.

Cette approche, garantit que les dépendances de données sont satisfaites avant chaque opération.

5.1.2 Gestion des registres virtuels

L'IR utilise un modèle à **assignation unique statique** simplifiée. Chaque résultat intermédiaire d'un calcul se voit attribuer un nouveau registre virtuel unique (`s0`, `s1`, ...) via un compteur monotone croissant. Cette stratégie simplifie énormément la génération d'IR : nous n'avons pas besoin de nous soucier de la réutilisation des registres à ce stade. C'est le Backend qui aura la charge de mapper cette infinité de registres virtuels vers les ressources finies de la machine.

5.2 Traduction des structures de contrôle

Les structures de contrôle de haut niveau (`While`, `IfThenElse`) n'existent pas dans notre IR linéaire. Elles doivent être traduites en sauts conditionnels et étiquettes.

5.3 Traduction des structures de contrôle

Les structures de contrôle de haut niveau (`While`, `IfThenElse`) n'existent pas dans notre IR linéaire. Elles doivent être traduites en sauts conditionnels et étiquettes.

5.3.1 Traduction optimisée du While

Pour l'instruction `while (condition) { body }`, nous avons opté pour une structure avec le test conditionnel placé à la fin du bloc. Cette approche réduit le nombre d'instructions de branchement exécutées à chaque itération.

La séquence générée est la suivante :

1. **Entrée :** Saut inconditionnel (`JMP`) vers l'étiquette d'évaluation de la condition (`E_Cond`).
2. **Corps :** Placement de l'étiquette de début (`E_Debut`) suivie du code généré pour le corps de la boucle.
3. **Condition :** Placement de l'étiquette `E_Cond`, suivie du calcul de l'expression conditionnelle.
4. **Bouclage :** Si la condition est vérifiée (c'est-à-dire différente de 0/false), un saut conditionnel (`BNE`) renvoie l'exécution vers `E_Debut`.
5. **Sortie :** Placement de l'étiquette de fin `E_Fin` pour la suite du programme.

Voici le schéma type de l'IR produite :

Schéma de génération (Pseudo-IR) :

```
    ; Saut initial vers le test
    JMP E_Cond.0

E_Debut.0:
    <Code du corps (body)>

E_Cond.0:
    <Calcul de la condition (cond)>
    ; Si cond est Vrai (!= 0), on boucle au début
    BNE_INT cond, #0, E_Debut.0

E_Fin.0:
    ; Suite du programme...
```

Cette transformation est robuste et permet l'imbrication arbitraire de boucles grâce à la gestion unique des identifiants d'étiquettes (le suffixe `.0`, `.1`, etc.).

5.4 Génération de code cible

C'est l'étape où la "magie" de l'abstraction s'efface pour affronter les contraintes matérielles.

5.4.1 Sélection d'instructions

Pour le backend RISC-V, nous effectuons une **macro-expansion** simple. La plupart des instructions IR ont un équivalent direct en RISC-V :

— `ADD_INT` devient `add`.

— `JMP` devient `j`.

Cependant, certaines opérations nécessitent plusieurs instructions machines. Par exemple, le chargement d'un immédiat entier (`LOAD_INT dst, #imm`) utilise la pseudo-instruction `li` (Load Immediate) de l'assembleur RISC-V, qui peut se décomposer en deux instructions (`lui` + `addi`) si la constante dépasse 12 bits.

5.4.2 Implémentation de la stratégie "All-on-Stack"

La stratégie d'allocation est déléguée aux classes `RVStackManager` et `RVDataManager`.

Calcul des Offsets (`RVStackManager`)

Avant de générer la moindre instruction, le `RVStackManager` parcourt les définitions de variables et les registres scratch utilisés dans l'`IRProgram`. Il construit un objet `RVOffsets` qui mappe chaque entité virtuelle à un offset relatif au pointeur de pile (`sp`). L'alignement de la pile à 16 octets (exigence de l'ABI RISC-V) est automatiquement géré ici.

Mouvements de données (`RVDataManager`)

Le `RVDataManager` simplifie l'écriture des visiteurs d'instructions. Par exemple, pour générer une addition, le visiteur demande simplement au DataManager de "charger l'opérande X dans `t0`". Le `RVDataManager` détermine dynamiquement la nature de l'opérande :

- Si c'est un **Immédiat** : Il génère un `li t0, val`.
- Si c'est un **Registre Virtuel / Variable** : Il récupère l'offset via le StackManager et génère un `lw t0, offset(sp)`.

Cette abstraction permettrait d'introduire facilement du *Spilling* sélectif : si le StackManager décidait de garder certaines variables dans des registres callee-saved (`s0-s11`), le DataManager générerait des instructions `mv` au lieu de `lw`, sans changer la logique du visiteur principal.

5.4.3 Différences entre Simulateur et Bare-Metal

Il est important de noter que notre backend génère du code pour le simulateur **RARS** et non pour un processeur RISC-V "nu" (Bare-Metal) ou sous Linux. Nous exploitons les services `ecall` de RARS pour les entrées/sorties et la terminaison du programme. Sur une cible réelle, il aurait fallu implémenter manuellement ces services (via l'UART ou des appels systèmes Linux `sys_write`), ce qui aurait considérablement alourdi la tâche sans apporter de valeur ajoutée à la démonstration de la modularité.

6 Environnement et Validation

L'introduction de nouvelles architectures cibles (RISC-V, QBE) et la refonte de la génération de code via une Représentation Intermédiaire nécessitent une méthodologie de validation rigoureuse. De plus, la diversité des outils requis (simulateurs, chaînes de compilation croisées) impose de standardiser l'environnement de développement pour garantir la reproductibilité des résultats au sein de l'équipe.

6.1 Environnement Docker

L'un des défis majeurs de cette extension était l'hétérogénéité des outils nécessaires. Les postes de travail de l'Ensimag ne disposent pas nativement des outils pour RISC-V (comme RARS) ou du backend QBE, et les droits d'administration restreints empêchent leur installation directe.

Pour contourner ce problème et assurer que "ce qui marche chez moi marche chez les autres", nous avons conteneurisé l'intégralité de notre environnement de développement et de test.

6.1.1 Construction de l'image

Nous avons défini un `Dockerfile` personnalisé basé sur une image `ubuntu:24.04`. Ce fichier automatise l'installation des dépendances complexes qui manquent sur les postes de l'école :

1. **Java 23 (Temurin) :** Installation manuelle du JDK 23 dans `/usr/local/jdk-23.0.2+7`, nécessaire pour les dernières fonctionnalités Java utilisées dans le projet.
2. **Compilation de QBE :** Le backend QBE n'étant pas disponible dans les dépôts standards, le Dockerfile télécharge les sources de la version 1.2, les compile via `make` et installe le binaire dans `/usr/local/qbe/bin`.
3. **Intégration de RARS :** Le simulateur `rars.jar` est copié dans l'image pour permettre l'exécution des tests RISC-V.

La construction de l'image s'effectue simplement via la commande standard :

```
$ docker build -t projetgl ./docker
```

6.1.2 Cycle de vie et Configuration

Pour faciliter l'usage, nous avons un script `run.sh` qui gère le redémarrage du conteneur persistant `projetgl`. Une fois dans le conteneur, l'environnement n'est pas encore totalement prêt : il faut lier les outils au projet courant.

C'est le rôle du script `bashrc` fourni dans le dossier `docker/`. Ce script effectue deux opérations critiques lorsqu'il est sourcé :

— **Résolution dynamique du chemin :** La fonction `get_gl_dir` calcule le chemin absolu de la racine du projet, peu importe où le dossier a été monté.

— **Injection du PATH :** Il ajoute les chemins vers les scripts de test (`src/test/script`) et les binaires compilés, rendant les commandes comme `decac` ou `test_etapeC.sh` accessibles globalement.

Le workflow typique d'un développeur de l'équipe est donc le suivant :

```
# 1. Entrée dans l'environnement conteneurisé
$ ./docker/run.sh

# 2. Activation des outils (normalement fait au build)
[gl@docker]$ source ./docker/bashrc

# 3. L'environnement est prêt
[gl@docker]$ decac -ir 1 test.deca  # Compilation vers RISC-V de ./src/test/*/test.deca
[gl@docker]$ rars test.s            # Exécution
```

6.2 Stratégie de test multiniveaux

La validation d'un compilateur modulaire ne peut se limiter à vérifier l'assembleur final. Si une erreur survient, il est crucial de savoir si elle provient de la génération de l'IR (MiddleEnd) ou de sa traduction en assembleur (BackEnd). Nous avons donc adopté une stratégie à deux niveaux.

6.2.1 Niveau 1 : Tests Unitaires de l'IR (White Box)

L'objectif est de valider la sémantique de l'IR indépendamment de toute architecture cible. Pour cela, nous avons développé un composant clé : l'**IRSimulator**.

Le concept de l'IRSimulator

L'`IRSimulator` est un interpréteur écrit en Java capable d'exécuter un `IRProgram` en mémoire. Il joue le rôle d'un "Backend Artificiel" de référence. Il simule le comportement de la machine abstraite définie par notre IR en utilisant :

— Une `Map<String, Object>` pour représenter les registres virtuels.
— Une `Map<Address, Object>` pour représenter la mémoire (Stack).
— Des flux de sortie capturés pour vérifier les `PRINT`.

Cela nous permet de tester unitairement chaque instruction IR. Si un test passe avec le simulateur mais échoue en RISC-V, nous savons immédiatement que le bug se situe dans le `RVGenerator`.

Implémentation des tests

Les tests sont regroupés dans le package `fr.ensimag.deca.ir` :
— `TestIRFloat` : Arithmétique flottante.
— `TestIRInt` : Arithmétique entière.
— `TestIRIntFloat` : Interactions entre types et conversions .
— `TestIRIO` : Vérification des sorties (`print`).
— `TestIRBasic` : Test d'ensemble.

Voici un exemple concret de test unitaire validant une instruction d'addition :

```java
@Test
public void testAddIntegers() {
    // Test: s0 = 15 + 27
    IRProgram program = new IRProgram();
    IRScratch s0 = program.scratch(IRType.INT);

    program.addInstruction(new IRBinaryExpr(null,
    "ADD", s0, new IRImmediateInt(15), new IRImmediateInt(27)
    ));
    program.addInstruction(new IRPrintValue(null, IRType.INT, s0));

    IRSimulator simulator = new IRSimulator(program);
    for (IRAbstractInst inst : program.text()) {
        inst.accept(simulator);
    }

    assertEquals("42", simulator.getOutput());
}
```

Listing 6.1 – Exemple de test unitaire IR (JUnit)

Cette méthode nous a permis de développer et stabiliser l'IR avant même d'avoir du générateur RISC-V complet.

6.2.2 Niveau 2 : Tests d'Intégration (Black Box)

Une fois l'IR validée, il faut s'assurer que la chaîne complète (Deca → IR → ASM → Exécution) fonctionne.

Adaptation des scripts existants

Nous avons modifié le script de validation de l'étape C (`test_etapeC.sh`) pour supporter le polymorphisme des cibles. Le script accepte désormais des arguments pour choisir le backend.

Le flux de test automatisé est le suivant :

1. Compilation du fichier `.deca` avec l'option `-ir` et la cible spécifiée (ex : `-r` pour RISC-V).
2. Génération du fichier assembleur correspondant (`.s` ou `.qbe`).
3. Exécution du code généré via le simulateur approprié (`rars` ou `qbe`).
4. Comparaison de la sortie standard avec le résultat attendu (oracle).

Filtrage du périmètre

Notre extension ne supportant pas encore les objets, le script de test a été configuré pour ne s'exécuter que sur le dossier `src/test/deca/codegen/valid/sans-objet/`. Cela évite les faux positifs liés aux fonctionnalités hors périmètre.

Exemple de commande de test d'intégration :

```
# Lancer tous les tests sans-objet sur le backend RISC-V
$ ./test_etapeC.sh -ir 1
[INFO] [Valides (Sortie Conforme)]
...
[SUCCESS]  Exécution conforme pour arith_full.deca
[SUCCESS]  Exécution conforme pour if.deca
...
```

6.3 Résultats

Les campagnes de tests confirment la validité de notre approche modulaire.

6.3.1 Couverture fonctionnelle

Dans la limite du périmètre d'implémentation défini (Deca Sans Objet), **90% des tests d'intégration valides passent** sur le backend RISC-V (les autres ne sont que des demos).

Cela inclut :

— L'arithmétique complexe (priorité des opérateurs, parenthèses).
— Les structures de contrôle imbriquées (While dans If, etc.).
— Les entrées/sorties basiques.
— L'invalidité de l'overflow de multiplication entière.

les 10% de fail sont les suivants :

— Des problématiques de stack-overflow spécifiques à RARS non rencontrées avec IMA.
— La manipulation de variable non initialisée est considérée comme valide avec RARS.

La cohérence des résultats entre le simulateur IR (Java), le simulateur IMA et le simulateur RARS (RISC-V) démontre la robustesse de la sémantique de notre Représentation Intermédiaire.

6.4 Limites de la validation

Bien que notre stratégie de test à deux niveaux (unitaire IR et intégration système) ait permis de garantir la correction fonctionnelle de l'extension, elle ne couvre pas l'intégralité du spectre de validation d'un compilateur industriel.

— **Validation sur simulateur uniquement :** La totalité des tests RISC-V a été effectuée sur le simulateur *RARS*. Bien que fidèle à la spécification ISA, ce simulateur ne reproduit pas certaines contraintes du matériel réel (latences mémoire, alignement strict, gestion du cache). Le code généré n'a pas été éprouvé sur une carte FPGA ou un processeur RISC-V physique.
— **Absence de métriques de performance :** Notre validation est actuellement binaire (le test passe ou échoue). Nous n'avons pas mis en place de *benchmarks* pour mesurer l'efficacité du code généré (temps d'exécution, empreinte mémoire, taille du binaire) comparé au backend IMA historique ou à un compilateur standard comme GCC.
— **Tests de charge (Stress Tests) :** Les programmes de test utilisés, issus de la suite de validation de base, restent de taille modeste. Le comportement du compilateur et des exécutables générés face à des cas limites — comme une récursion très profonde (Test de Stack Overflow) ou des fichiers sources très volumineux — n'a pas été extensivement testé sur les nouvelles cibles.
— **Dépendance à l'environnement RARS :** Le code assembleur généré utilise les services spécifiques fournis par le simulateur RARS (les *Environment Calls*). Ce code ne pourrait pas s'exécuter tel quel sur un processeur RISC-V physique tournant sous Linux ou en "Bare Metal". Pour cela, il faudrait réécrire la gestion des entrées/sorties pour utiliser les appels systèmes standard du noyau Linux (ABI Linux/RISC-V) et gérer manuellement l'initialisation du runtime (`_start`). Nous avons volontairement écarté cette complexité pour nous concentrer sur la justesse de la traduction IR → ASM.

Ces limites soulignent que si la *faisabilité* de l'extension est prouvée, sa *robustesse* en conditions réelles d'exécution nécessiterait une phase de validation supplémentaire.

7 Conclusion et Perspectives

Ce projet d'extension a permis de transformer le compilateur `decac`, initialement monolithique et lié à une architecture pédagogique spécifique, en une chaîne de compilation modulaire et ouverte sur les architectures standards de l'industrie.

7.1 Bilan technique

L'objectif principal était de démontrer la viabilité d'une architecture articulée autour d'une Représentation Intermédiaire (IR). Cet objectif est atteint. L'introduction de notre IR de type *Three-Address Code* a permis de briser le couplage fort entre l'Analyse Syntaxique (Frontend) et la Génération de Code (Backend).

La réussite technique la plus probante est la coexistence fonctionnelle de trois backends radicalement différents :

1. **IMA :** Une machine CISC à registres limités et opérandes mémoire.
2. **RISC-V :** Une machine Load/Store moderne à registres nombreux.
3. **QBE :** Un langage intermédiaire textuel de bas niveau.

Le fait que le même code source Deca (sans objet) produise des exécutions identiques sur ces trois cibles valide la sémantique de notre IR. Nous avons réussi à abstraire les concepts de haut niveau du langage (boucles `while`, conditions) en une séquence linéaire d'instructions atomiques, résolvant ainsi le problème « $M \times N$ » de la compilation.

7.2 Bilan pédagogique

Au-delà du code produit, cette extension a été vecteur d'apprentissages majeurs pour l'équipe :

— **Architecture des Processeurs :** Nous avons dû comprendre en profondeur les différences entre une architecture CISC et RISC. La gestion explicite de la pile, des conventions d'appel (ABI) et des registres physiques sur RISC-V nous a donné une vision concrète du fonctionnement bas niveau d'un ordinateur, dépassant le cadre théorique des cours.

— **Design Patterns :** L'implémentation massive du patron de conception *Visitor* (pour la génération d'IR depuis l'AST, puis pour la génération d'assembleur depuis l'IR) a renforcé notre maîtrise de la programmation orientée objet en Java. Nous avons compris l'intérêt du *Double Dispatch* pour séparer les algorithmes des structures de données.

— **Travail en parallèle :** La définition d'une interface stricte pour l'IR a permis à l'équipe de travailler simultanément sur le Frontend (génération de l'IR) et les Backends, simulant ainsi un environnement de développement industriel.

7.3 Perspectives d'évolution

Si l'architecture modulaire est en place, le compilateur actuel reste limité au sous-ensemble procédural du langage. Plusieurs axes d'amélioration sont identifiés pour en faire un compilateur de production.

7.3.1 Support de Deca Objet et VTables

L'étape logique suivante est l'implémentation de la Programmation Orientée Objet. Dans une IR linéaire, cela implique de :

— Gérer l'allocation dynamique dans le tas (segment `.heap`).

— Implémenter le dispatch dynamique des méthodes. Cela nécessiterait l'ajout de structures de données statiques pour les **VTables** (Tables de Méthodes Virtuelles) et de nouvelles instructions IR pour le chargement de pointeurs de fonction (ex : `LOAD_METHOD`).

7.3.2 Optimisations sur l'IR

L'un des avantages majeurs du 3AC est la facilité d'analyse du flot de données. Actuellement, notre compilateur effectue une traduction naïve. Nous pourrions implémenter des passes d'optimisation entre la génération d'IR et les backends :

— **Constant Folding :** Simplifier les expressions constantes (ex : `3 + 4` devient `7`) directement dans l'IR.

— **Élimination du code mort :** Supprimer les blocs d'instructions après un `return` ou un saut inconditionnel.

— **Allocation de registres intelligente :** Remplacer notre mapping naïf par un algorithme plus complexe afin de minimiser les accès mémoire.

7.3.3 Évolutivité du Backend RISC-V

Bien que notre implémentation actuelle de la génération assembleur soit fonctionnelle mais sous-optimale (stratégie "All-on-Stack"), l'architecture logicielle mise en place est prête pour l'avenir. Grâce à la décomposition en *Managers* (`RVStackManager`, `RVDataManager`), le backend RISC-V est conçu pour être étendu. Un expert en optimisation bas niveau pourrait aisément :

— Remplacer le `RVStackManager` naïf par un allocateur de registres basé sur la coloration de graphe.

— Raffiner le `RVDataManager` pour maintenir les variables les plus utilisées dans les registres callee-saved (`s0-s11`) au lieu de la pile.

Cette dette technique est donc maîtrisée et encapsulée, ne remettant pas en cause la structure globale du compilateur.

Bibliographie

[1] Quentin Carbonneaux and the community :`https://c9x.me/compile/`

[2] Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanović. *The RISC-V Instruction Set Manual, Volume I : Base User-Level ISA* `https://riscv.org/technical/specifications/`

[3] *Projet Génie Logiciel : 2e année*. Grenoble INP - Ensimag, Décembre 2025.

[4] GeeksforGeeks. *Three Address Code in Compiler Design*. `https://www.geeksforgeeks.org/three-address-code-compiler/`

[5] The RARS Team. *RARS : RISC-V Assembler and Runtime Simulator*. `https://github.com/TheThirdOne/rars`